# Profiling-Guided Optimization of a C-Based Movie Recommendation System

Scientific Computing Assignment 2

April 25, 2026

**Abstract**

This report presents a profiling-guided optimization of a C-based movie recommendation system. The original implementation uses collaborative filtering with Pearson correlation, K-Means clustering, prediction generation, and sorting to recommend movies to a target user. The goal of the assignment was to reduce the average recommendation time without introducing a new recommendation algorithm. The optimization process focused on code-level and computational improvements such as compiler optimization, reducing repeated file I/O, caching loaded data structures, improving sorting, and safely applying OpenMP only to suitable numerical loops. The final optimized implementation achieved a warm-run speedup of approximately 12.75× while preserving the correctness of the Top-10 recommendation output.

## 1 Introduction

Recommendation systems use previous user ratings or preferences to recommend unseen items to a target user. In the given assignment, the data is represented as a user-item preference matrix, and recommendations are generated by comparing user vectors, finding similar users, and predicting movies that the target user may prefer.

The provided Movie Recommendation System is a rudimentary C implementation based on collaborative filtering. The main objective of this work was to execute the original system, measure its runtime, identify performance bottlenecks using profiling tools, and optimize the average time required to generate a recommendation.

The key constraint followed throughout this work was that the recommendation algorithm itself was not replaced. Instead, the same system was optimized through implementation-level improvements.

## 2 Execution Environment

The experiments were performed in the following environment.

Table 1: Execution environment

| Component | Configuration |
|---|---|
| Operating System | Ubuntu 24.04, Linux 6.8.0 x86_64 |
| CPU | Intel(R) Xeon(R) Processor @ 2.30GHz |
| Compiler | GCC 13.3.0 |
| OpenMP Threads | 4 threads |
| Baseline Flags | `-O0 -lm` |
| Optimized Single-Thread Flags | `-O3 -march=native -lm` |
| Optimized OpenMP Flags | `-O3 -march=native -fopenmp -lm` |

## 3 Dataset Configuration

The dataset was analyzed using a Python script, `scripts/analyze_dataset.py`. The file used for benchmarking was `Dataset/ratings_learn.csv`.

Table 2: Dataset summary

| Metric | Value |
|---|---|
| Unique Users | 548 |
| Unique Movies | 8297 |
| Total Ratings | 80963 |
| User-Movie Matrix Size | 4,546,756 entries |
| Density | 1.78% |
| Sparsity | 98.22% |

The high sparsity indicates that most user-movie pairs do not contain ratings. This is typical in recommender systems and makes repeated full-matrix processing inefficient.

## 4 Baseline System

The baseline system uses the following major components:

- Utility matrix construction from rating data.
- Matrix normalization.
- Pearson correlation for user similarity.
- K-Means clustering.
- Prediction generation.
- Sorting of predicted movie ratings.

In the baseline version, each recommendation call performs repeated loading and processing of data. This causes both cold and warm runs to behave similarly, because the system does not effectively reuse already loaded structures.

## 5 Profiling Methodology

Two profiling and timing methods were used:

- **gprof**: Used to identify function-level hotspots.
- **Wall-clock timing**: Used through `gettimeofday()` and `omp_get_wtime()` to measure cold and warm recommendation times.

The profiling process followed a simple performance tuning cycle:

$$\text{Measure} \rightarrow \text{Analyze} \rightarrow \text{Optimize} \rightarrow \text{Re-measure}$$

The system was benchmarked using three builds:

1. Baseline unoptimized build.
2. Optimized single-threaded build.
3. Optimized OpenMP build.

## 6 Bottlenecks Identified

The `gprof` output showed that the main runtime cost was concentrated in matrix normalization, Pearson similarity calculation, utility matrix creation, and similarity computation.

Table 3: Main profiling hotspots

| **Function** | **Time %** | **Calls** | **Reason** |
|---|---|---|---|
| `normalize_matrix` | 46.67% | 1 | Matrix normalization cost |
| `pearson_correlation` | 26.67% | 548 | Repeated user similarity calculation |
| `get_utility_matrix` | 13.33% | 1 | Utility matrix construction |
| `calc_average` | 6.67% | 548 | Repeated average calculation |
| `calc_similarity` | 6.67% | 1 | Calls Pearson correlation repeatedly |

The key bottlenecks were:

1. **Repeated file I/O**: Data files were read repeatedly during recommendation calls.
2. **Repeated matrix processing**: Normalization and similarity calculations were recomputed unnecessarily.
3. **Inefficient sorting**: Full sorting was used even though only the highest ranked recommendations were needed.
4. **Unsafe parallel overhead**: Early OpenMP usage in setup or initialization paths introduced overhead and unstable cold-run behavior.

# 7 Optimization Strategies

## 7.1 Compiler Optimization

The optimized versions were compiled using:

```
-O3 -march=native
```

This allows GCC to apply aggressive optimization, instruction selection, loop optimizations, and CPU-specific improvements.

## 7.2 Data Caching

The most important optimization was avoiding repeated file loading and parsing. The optimized version caches the required dataset structures after the first recommendation call. Therefore, the first call is treated as a *cold run*, while the second call is treated as a *warm run*.

$$\text{Cold Run} = \text{Initialization} + \text{Data Loading} + \text{Recommendation}$$

$$\text{Warm Run} = \text{Cached Data} + \text{Recommendation}$$

This separation gives a fairer explanation of where the performance improvement comes from.

## 7.3 Pearson Similarity Optimization

The Pearson similarity calculation was improved by reducing repeated calculations and avoiding unnecessary repeated work inside inner loops. Since Pearson correlation is called many times, even small improvements in this function can reduce total runtime.

## 7.4 Top-K Sorting Optimization

The baseline implementation sorted a large list of predictions even though the final output only required the top recommendations. The optimized implementation reduces unnecessary sorting work by focusing on the top-ranked candidates.

This reduces wasted comparisons and improves runtime during recommendation generation.

## 7.5 OpenMP Optimization

OpenMP was applied only after auditing the code for safe parallel execution. Unsafe OpenMP pragmas were removed from file I/O, memory allocation, initialization, and small loops.

OpenMP was retained only for large independent numerical loops where:

- loop iterations are independent,
- there is no file I/O inside the loop,
- there are no unsafe shared writes,
- thread-local or reduction variables are used when needed.

This avoids race conditions and unnecessary thread overhead.

# 8 Correctness Verification

Correctness was verified by comparing the final Top-10 recommendations produced by:

- baseline build,
- optimized single-thread build,
- optimized OpenMP build.

The correctness script compared the final recommendation outputs and confirmed that the optimized versions preserved the same recommendation results.

Table 4: Correctness verification result

| **Comparison** | **Result** |
|---|---|
| Baseline vs Optimized Single-Thread | PASS |
| Baseline vs Optimized OpenMP | PASS |

This confirms that the optimization did not introduce meaningful correctness regressions.

# 9 Benchmark Results

## 9.1 Cold Run Results

The cold run measures the first recommendation call, including data loading and initialization.

Table 5: Cold run benchmark results

| **Version** | **Cold Avg Time** | **Speedup** | **Notes** |
|---|---|---|---|
| Baseline `-O0` | 0.90s | 1.00× | Reference implementation |
| Optimized Single-Thread `-O3` | 0.50s | 1.80× | Stable optimized version |
| Optimized OpenMP `-O3 -fopenmp` | 0.54s | 1.66× | Slight thread overhead in cold path |

The optimized single-threaded version performed best in the cold run. The OpenMP version was slightly slower than the single-threaded optimized version because the cold run still includes initialization overhead where parallelism provides limited benefit.

## 9.2 Warm Run Results

The warm run measures a second recommendation call in the same process, after data structures are cached.

Table 6: Warm run benchmark results

| **Version** | **Warm Avg Time** | **Speedup** | **Notes** |
|---|---|---|---|
| Baseline `-O0` | 0.88s | 1.00× | Reloads/reprocesses data |
| Optimized Single-Thread `-O3` | 0.069s | 12.75× | Caching + compiler optimization |
| Optimized OpenMP `-O3 -fopenmp` | 0.069s | 12.75× | Similar to single-thread on this dataset |

The warm run shows the largest improvement. This is because repeated file I/O and repeated initialization are avoided.

### 9.3 Graphical Results

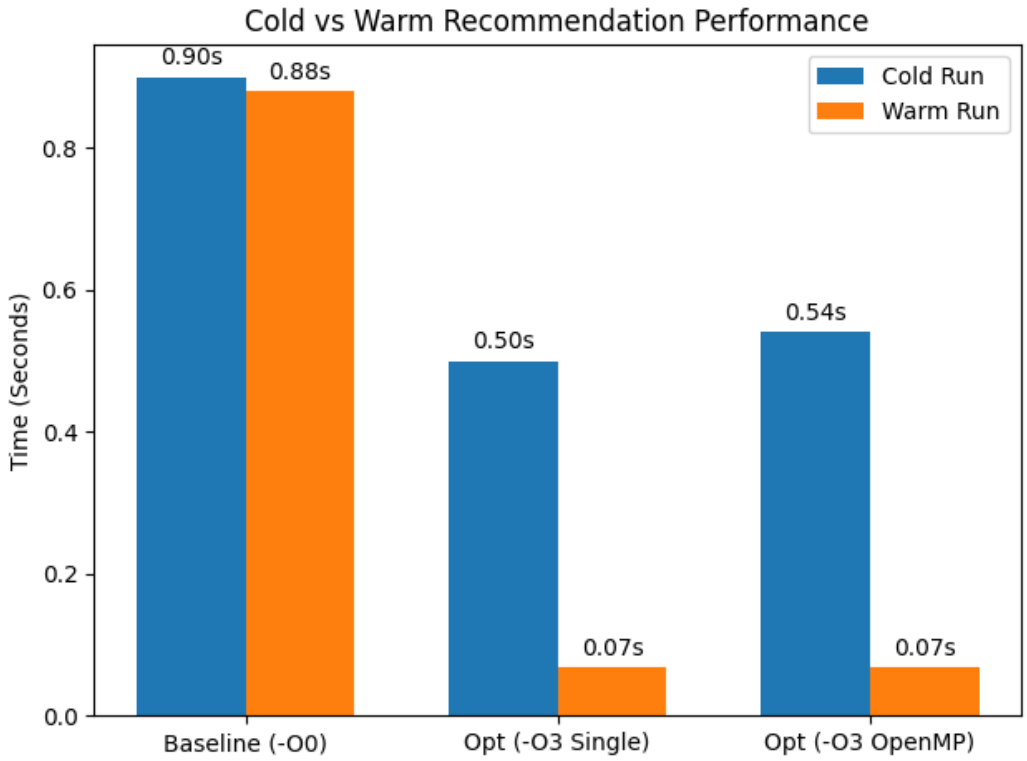


Figure 1: Cold and warm runtime comparison across baseline, optimized single-thread, and optimized OpenMP builds.

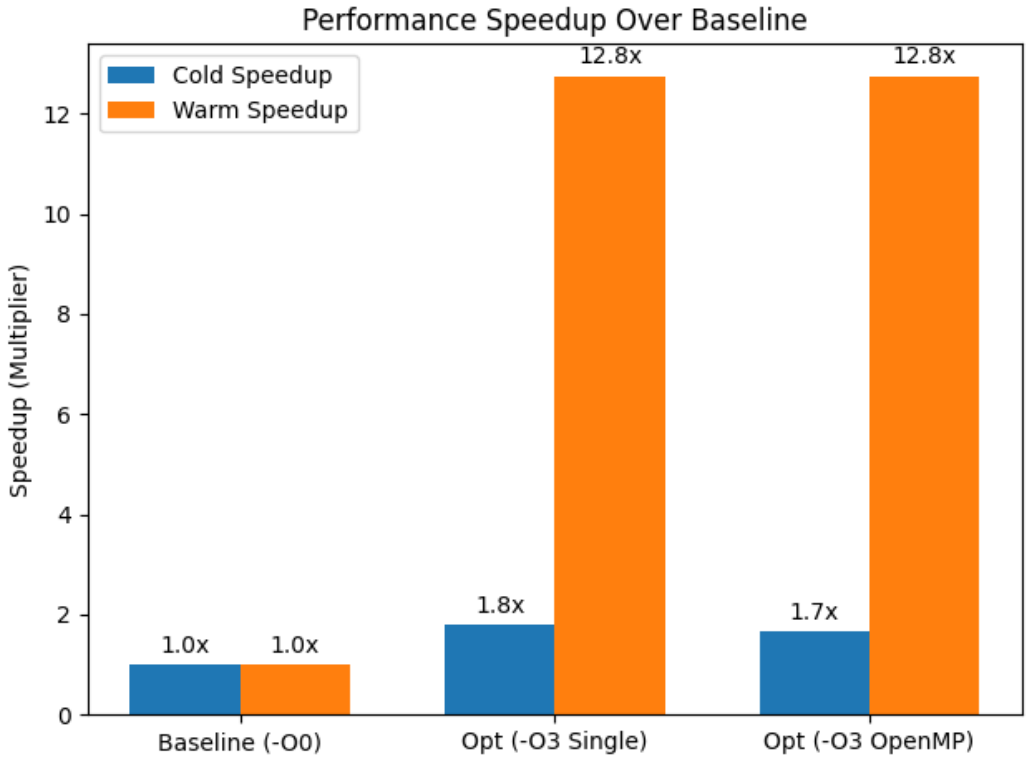


Figure 2: Speedup comparison for cold and warm recommendation runs.

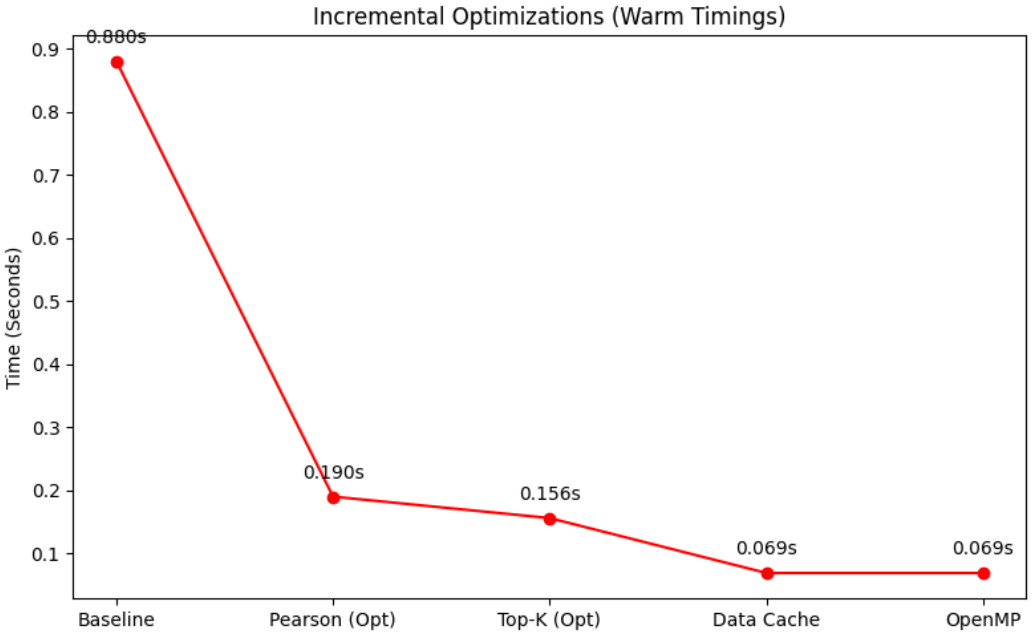


Figure 3: Incremental runtime improvement across optimization stages.

## 10 Discussion

The results show that the largest improvement comes from avoiding repeated data loading and repeated processing. In the baseline version, warm runs do not improve significantly because the required data is repeatedly loaded or recomputed. After caching is introduced, repeated recommendation calls become much faster.

The cold run improvement is smaller because it still includes initialization and loading overhead. The optimized single-threaded version achieved the best cold performance because it avoids OpenMP thread startup overhead.

OpenMP did not provide additional speedup over the optimized single-threaded warm run. This is mainly because the dataset is relatively small and sparse, with only 1.78% density. After caching and compiler optimization, the remaining computation is not large enough for OpenMP to provide significant additional benefit. In this case, thread scheduling overhead cancels out much of the expected parallel gain.

Therefore, the optimized single-threaded build is the most stable final version, while the OpenMP build is retained as an experimental parallel version that preserves correctness.

## 11 Conclusion

This project successfully optimized the given C-based movie recommendation system using profiling-guided computational improvements. The algorithmic structure of the recommender was preserved, and no new recommendation algorithm was introduced.

The main optimizations were:

- compiler optimization using `-O3 -march=native`,
- caching loaded data structures,
- reducing repeated computation in Pearson similarity,
- improving Top-K recommendation selection,
- safely auditing and limiting OpenMP usage.

The final optimized implementation achieved:

- **Cold speedup:** approximately 1.80× using the optimized single-thread build,
- **Warm speedup:** approximately 12.75× using the optimized cached build,
- **Correctness:** Top-10 recommendation output verified as matching the baseline.

Overall, the final result demonstrates that careful profiling, caching, and code-level optimization can significantly reduce recommendation time while preserving the original recommender behavior.